COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:		**SECTION:**	CPE21S1
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	Jarel Dave R. Egar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

Modify the given operations used in the singly linked lists to work on the new construct of a doubly linked list. Provide a screenshot of your code and analysis in table 3-4.

Screenshot	Main.cpp

Egar_HOA3pmain.cpp | Double_LL.h

Egar_HOA3p1 > SLL > Egar_HOA3pmain.cpp > main()

```cpp
#include <iostream>
#include "Double_LL.h"

int main() {
    // 'C' 'P' 'E'

    // head node
    DoubleList<char>* head = new_Node('C');

    // second node
    DoubleList<char>* second = new_Node('P');

    // third node
    DoubleList<char>* third = new_Node('E');

    // link first node with the second node
    head->next = second;
    second->prev = head;

    // link second node with the third node
    second->next = third;
    third->prev = second;

    std::cout << "Initial Traversal" << std::endl;
    DllTraverse(head);

    // Insert 'G' at the head
    std::cout << "Inserting 'G' at Head" << std::endl;
    DllInsertHead('G', head);
    DllTraverse(head);

    // Insert 'G' at the end
    std::cout << "Inserting 'G' at End" << std::endl;
    DllInsertEnd('G', head);
    DllTraverse(head);

    // Insert 'B' inside the list after 'P' (second node)
    std::cout << "General Insertion ('B' after 'P')" << std::endl;
    DllgeneralInsert('B', second);
    DllTraverse(head);

    // Delete a specific node ('J')
    std::cout << "Deleting node 'J'" << std::endl;
    DllDelete('J', head);
    DllTraverse(head);

    // Clean up memory by deleting the entire list
    std::cout << "Deleting Entire List" << std::endl;
    DllDeleteList(head);

    if (head == nullptr) {
        std::cout << "List successfully deleted and head is null" << std::endl;
    }

    return 0;
}
```

Double_LL.h

Egar_HOA3p1 > SLL > C Double_LLh > ...

```h
#ifndef Double_LL_H
#define Double_LL_H

#include <iostream>

template <typename T>
class DoubleList {
    public:
        T data;
        DoubleList<T>* next = nullptr;
        DoubleList<T>* prev = nullptr;
};

//----------------------
// Creating a node
//----------------------
template <typename T>
DoubleList<T>* new_Node(T newData) {
    // dynamically allocate memory for a new node
    DoubleList<T>* newNode = new DoubleList<T>;

    // store the data in the node
    newNode->data = newData;

    // since this is a new node, it does not point to any other node
    newNode->next = nullptr;
    newNode->prev = nullptr;
    return newNode;
}
```

```h
//----------------------
// Traversal
//----------------------
template <typename T>
void DllTraverse(DoubleList<T>* currentNode) {
    DoubleList<T>* tail = nullptr;

    // 1. Forward Traversal
    std::cout << "Forward Traversal:\n";
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << "\n"; // Newline so Reverse Traversal starts on a new line

    // 2. Reverse Traversal
    std::cout << "Reverse Traversal:\n";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << "\n\n"; // Newlines to space out calls from main()
}
```

```h
//----------------------
// Insertion at head
//----------------------
template <typename T>
void DllInsertHead(T newData, DoubleList<T>*& currenthead) {
    DoubleList<T>* newNode = new_Node(newData);

    newNode->next = currenthead;
    newNode->prev = nullptr;
    if (currenthead != nullptr) {
        currenthead->prev = newNode;
    }
    currenthead = newNode;
}
```

```cpp
//----------------------
// General Insertion
//----------------------
template <typename T>
void DllgeneralInsert(T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null.\n";
        return;
    }

    DoubleList<T>* newNode = new_Node(newData);

    // Link the new node
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    // If there is a node after prevNode, fix its prev pointer
    if (prevNode->next != nullptr)
        prevNode->next->prev = newNode;

    // Finish the insertion
    prevNode->next = newNode;
}
```

```cpp
// ----------------------
// Insertion at the end
// ----------------------
template <typename T>
void DllInsertEnd(T newData, DoubleList<T>*& currenthead) {
    // create a new node
    DoubleList<T>* newNode = new_Node(newData);

    // If the list is empty, new node becomes the head
    if (currenthead == nullptr) {
        currenthead = newNode;
        return;
    }

    // traverse using a temporary pointer until the last node
    DoubleList<T>* tail = currenthead;
    while (tail->next != nullptr) {
        tail = tail->next;
    }

    // connect the last node to the new node
    tail->next = newNode;

    // connect the prev node to the new node
    newNode->prev = tail;
}
```

```cpp
//----------------------
// Deletion
//----------------------

// Deletes a single node by value
template <typename T>
void DllDelete(T findData, DoubleList<T>*& head) {
    if (head == nullptr)
        return;

    DoubleList<T>* currNode = head;

    // Search for the node containing findData
    while (currNode != nullptr && currNode->data != findData)
        currNode = currNode->next;

    // Node not found
    if (currNode == nullptr)
        return;

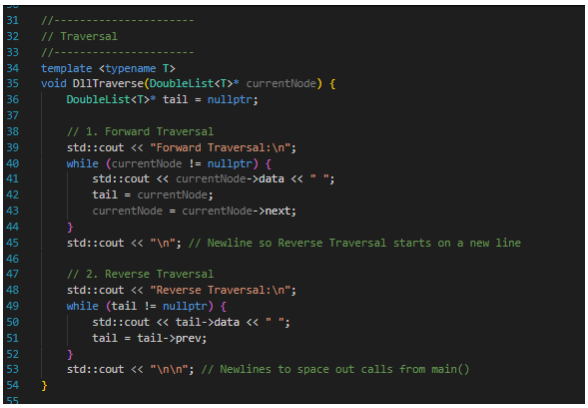
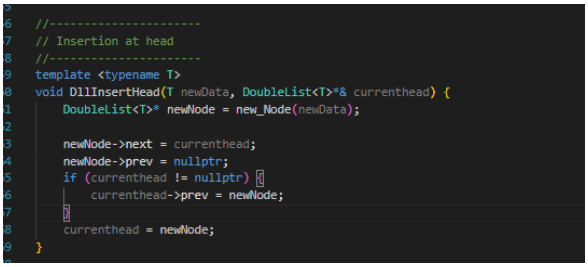
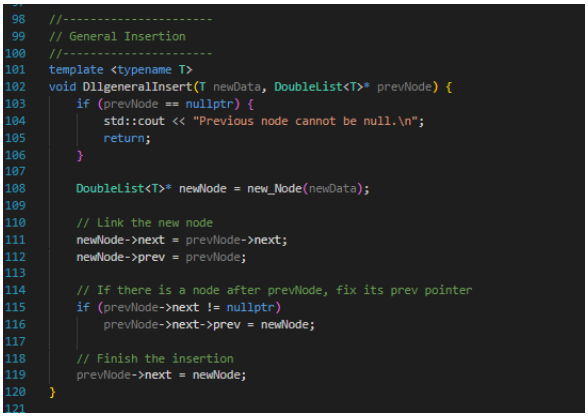
    // Update previous node's next pointer or move head pointer
    if (currNode->prev != nullptr)
        currNode->prev->next = currNode->next;
    else
        head = currNode->next;

    // Update next node's prev pointer
    if (currNode->next != nullptr)
        currNode->next->prev = currNode->prev;

    delete currNode;
}
// Deletes the entire list to free memory
template <typename T>
void DllDeleteList(DoubleList<T>*& head) {
    DoubleList<T>* currNode = head;

    while (currNode != nullptr) {
        DoubleList<T>* temp = currNode;
        currNode = currNode->next;
        delete temp;
    }
    head = nullptr;
}
```

Discussion	This code demonstrates how a doubly linked list works. Basically this is the same as the previous HOA which is the singly linked list, but the main difference is that a doubly linked list contains an additional pointer which is the prev pointer which points to the preceding node in the list. While a singly linked list only allowed me to move forward from head to tail, this prev pointer gives me the ability to travel both forward and back. However, it also required me to be extra careful when updating both next and prev links during insertions and deletions.

Operation	Screenshot
Traversal	
Insert at Head	
Insertion at any part	

```cpp
//----------------------
// Traversal
//----------------------
template <typename T>
void DllTraverse(DoubleList<T>* currentNode) {
    DoubleList<T>* tail = nullptr;

    // 1. Forward Traversal
    std::cout << "Forward Traversal:\n";
    while (currentNode != nullptr) {
        std::cout << currentNode->data << " ";
        tail = currentNode;
        currentNode = currentNode->next;
    }
    std::cout << "\n"; // Newline so Reverse Traversal starts on a new line

    // 2. Reverse Traversal
    std::cout << "Reverse Traversal:\n";
    while (tail != nullptr) {
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << "\n\n"; // Newlines to space out calls from main()
}
```

```cpp
//----------------------
// Insertion at head
//----------------------
template <typename T>
void DllInsertHead(T newData, DoubleList<T>*& currenthead) {
    DoubleList<T>* newNode = new_Node(newData);

    newNode->next = currenthead;
    newNode->prev = nullptr;
    if (currenthead != nullptr) {
        currenthead->prev = newNode;
    }
    currenthead = newNode;
}
```

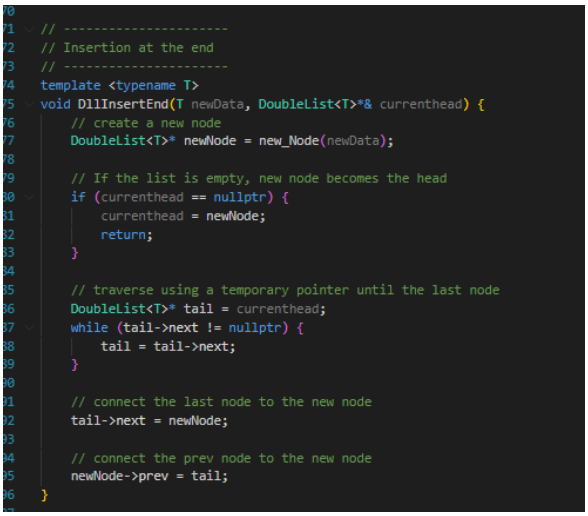
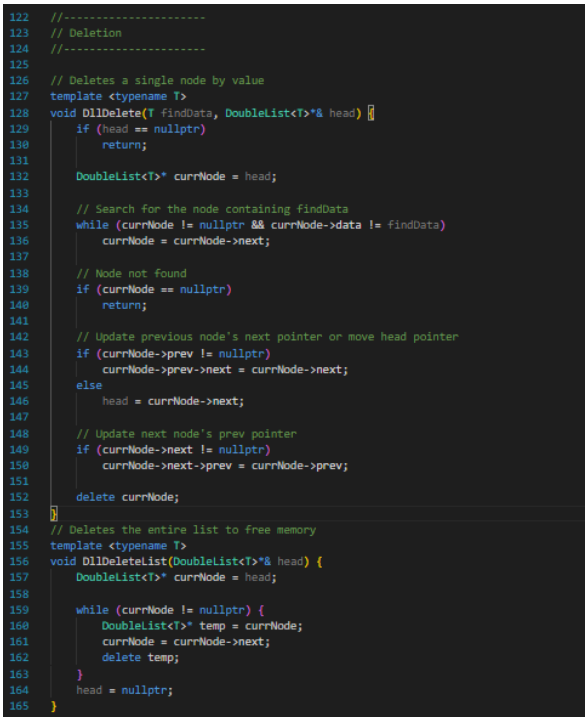
```cpp
//----------------------
// General Insertion
//----------------------
template <typename T>
void DllgeneralInsert(T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null.\n";
        return;
    }

    DoubleList<T>* newNode = new_Node(newData);

    // Link the new node
    newNode->next = prevNode->next;
    newNode->prev = prevNode;

    // If there is a node after prevNode, fix its prev pointer
    if (prevNode->next != nullptr)
        prevNode->next->prev = newNode;

    // Finish the insertion
    prevNode->next = newNode;
}
```

Insertion at the end	
Deletion of Node	

```cpp
// ----------------------
// Insertion at the end
// ----------------------
template <typename T>
void DllInsertEnd(T newData, DoubleList<T>*& currenthead) {
    // create a new node
    DoubleList<T>* newNode = new_Node(newData);

    // If the list is empty, new node becomes the head
    if (currenthead == nullptr) {
        currenthead = newNode;
        return;
    }

    // traverse using a temporary pointer until the last node
    DoubleList<T>* tail = currenthead;
    while (tail->next != nullptr) {
        tail = tail->next;
    }

    // connect the last node to the new node
    tail->next = newNode;

    // connect the prev node to the new node
    newNode->prev = tail;
}
```

```cpp
//----------------------
// Deletion
//----------------------

// Deletes a single node by value
template <typename T>
void DllDelete(T findData, DoubleList<T>*& head) {
    if (head == nullptr)
        return;

    DoubleList<T>* currNode = head;

    // Search for the node containing findData
    while (currNode != nullptr && currNode->data != findData)
        currNode = currNode->next;

    // Node not found
    if (currNode == nullptr)
        return;

    // Update previous node's next pointer or move head pointer
    if (currNode->prev != nullptr)
        currNode->prev->next = currNode->next;
    else
        head = currNode->next;

    // Update next node's prev pointer
    if (currNode->next != nullptr)
        currNode->next->prev = currNode->prev;

    delete currNode;
}
// Deletes the entire list to free memory
template <typename T>
void DllDeleteList(DoubleList<T>*& head) {
    DoubleList<T>* currNode = head;

    while (currNode != nullptr) {
        DoubleList<T>* temp = currNode;
        currNode = currNode->next;
        delete temp;
    }
    head = nullptr;
}
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++
Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing
Problem Description:
In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to

customize default implementations, such as when looping songs in a music player or in games where multiple Players take a turn one by one in a circle.
These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.
We'll take the use case of a music player. It should have following functions supported:
Create a playlist using multiple songs.
24
Add songs to the playlist.
Remove a song from the playlist.
Play songs in a loop (for this activity, we will print all the songs once).
Here are the steps to solve the problem:
Design the basic structure that supports circular data representation.
After that, implement the insert and delete functions in the structure.
Implement a function for traversing the playlist.
The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

MAIN.CPP

Egar_mainsupple.cpp | Circular_LL.h

Egar_Supplementray3.1 > Egar_mainsupple.cpp > main()

```cpp
#include <iostream>
#include "Circular_LL.h"

int main(){

    //creating the playlist head
    SingleSongNode<std::string>* playlist = nullptr;

    //add songs
    std::cout<<"\n";

    addSong(std::string("Song 1"), playlist);
    addSong(std::string("Song 2"), playlist);
    addSong(std::string("Song 3"), playlist);

     std::cout<<"\n";

    NextSong(playlist);
    PrevSong(playlist);

    std::cout<<"\n";

    std::cout<<"Now playing: \n";
    PlayAll(playlist);

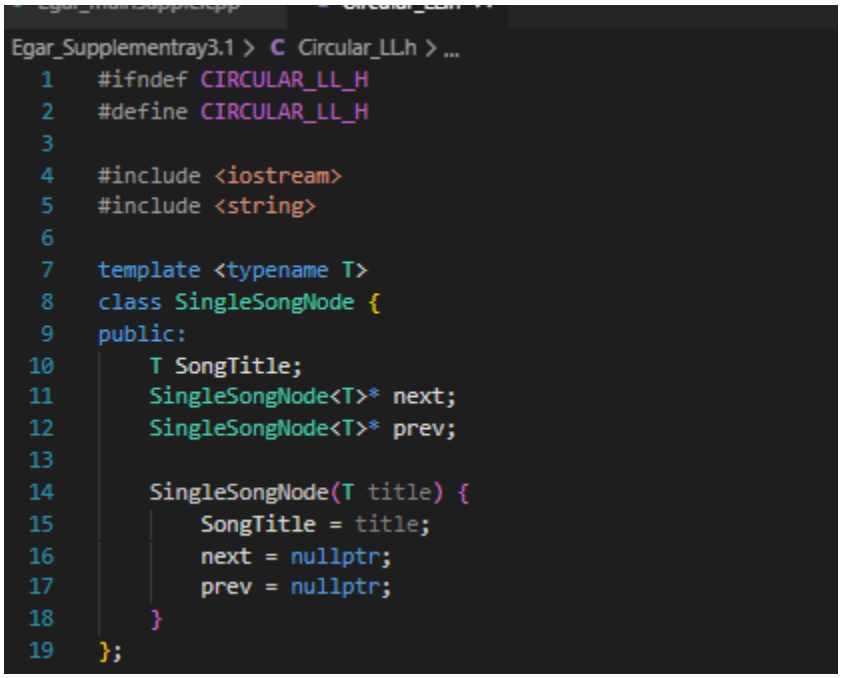
    std::cout<<"\n";
    DeleteSong(std::string("Song 2"), playlist);
    std::cout<<"\n";

    std::cout<<"Now playing: \n";
    PlayAll(playlist);


 return 0;
}

```

Circular_LL.h




```h
#ifndef CIRCULAR_LL_H
#define CIRCULAR_LL_H

#include <iostream>
#include <string>

template <typename T>
class SingleSongNode {
public:
    T SongTitle;
    SingleSongNode<T>* next;
    SingleSongNode<T>* prev;

    SingleSongNode(T title) {
        SongTitle = title;
        next = nullptr;
        prev = nullptr;
    }
};
```

```cpp
// Add Songs
template <typename T>
void addSong(T newTitle, SingleSongNode<T>*& PlayListHead) {

    SingleSongNode<T>* newNode = new SingleSongNode<T>(newTitle);

   // this checks if the playlist is empty, then the new song will point to itself
    if (PlayListHead == nullptr) {
        PlayListHead = newNode;
        PlayListHead->next = PlayListHead;
        PlayListHead->prev = PlayListHead;
        std::cout << "Add initial song: " << newTitle << "\n";
        return;
    }

    // Insert at end (circular)
    SingleSongNode<T>* tail = PlayListHead->prev;

    tail->next = newNode;
    newNode->prev = tail;

    newNode->next = PlayListHead;
    PlayListHead->prev = newNode;

    std::cout << "Added song: " << newTitle << "\n";
}
```

```cpp
// Next Song
template <typename T>
void NextSong(SingleSongNode<T>*& PlayListHead) {
    if (PlayListHead == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }
    PlayListHead = PlayListHead->next; // this checks if the playlist is empty, then the new song will point to itself
    std::cout << "Now playing: " << PlayListHead->SongTitle << "\n";
}

// Prev Song | same function as next song but it moves to the previous song in the playlist
template <typename T>
void PrevSong(SingleSongNode<T>*& PlayListHead) {
    if (PlayListHead == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }
    PlayListHead = PlayListHead->prev; //responsible for moving to the previous song in the playlist
    std::cout << "Now playing: " << PlayListHead->SongTitle << "\n";
}
```

```cpp
// Play All
template <typename T>
void PlayAll(SingleSongNode<T>*& PlayListHead) {
    if (PlayListHead == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }

    SingleSongNode<T>* current = PlayListHead; //creates a pointer for the head of the playlist and traverse each node to find songs

    do {
        std::cout << "Now playing: " << current->SongTitle << "\n";
        current = current->next;
    } while (current != PlayListHead);
}
```

```cpp
// Delete Song
template <typename T>
void DeleteSong(T title, SingleSongNode<T>*& PlayListHead) {

    if (PlayListHead == nullptr) {
        std::cout << "Playlist is empty.\n";
        return;
    }

    SingleSongNode<T>* current = PlayListHead;

    do {
        if (current->SongTitle == title) {

            // if only one node
            if (current->next == current) {
                delete current;
                PlayListHead = nullptr;
                std::cout << "Deleted song: " << title << "\n";
                return;
            }

            // Removing of node
            current->prev->next = current->next;
            current->next->prev = current->prev;

            // Deleting head
            if (current == PlayListHead) {
                PlayListHead = current->next;
            }
```

```
117             delete current;
118             std::cout << "Deleted song: " << title << "\n";
119             return;
120         }
121
122         current = current->next;
123
124     } while (current != PlayListHead);
125
126     std::cout << "Song not found.\n";
127 }
128
129 #endif
```

OUTPUT

```
PS D:\VSCode\Projects> cd "d:\VSCode\Projects\Egar_Supplementray3.1\" ; if ($?) { g++ Egar_mainsupple.cpp -o Eg
ar_mainsupple } ; if ($?) { .\Egar_mainsupple }

Add initial song: Song 1
Added song: Song 2
Added song: Song 3

Now playing: Song 2
Now playing: Song 1

Now playing:
Now playing: Song 1
Now playing: Song 2
Now playing: Song 3

Deleted song: Song 2
Now playing:
Now playing: Song 1
Now playing: Song 3
PS D:\VSCode\Projects\Egar_Supplementray3.1>
```

Discussion and code analysis:

This activity helped me understand how a circular linked list works through the use of nodes and pointers. In order for the playlist to function properly, it is necessary to use operations such as adding songs, deleting songs, and navigating to the next and previous songs. Because of this lesson, I learned that a circular linked list allows continuous traversal since the last node connects back to the first node, and I also realized that proper pointer handling is important to maintain the structure and avoid errors.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **Chat gpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explaining the error**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I asked chatgpt to provide a detailed explanation and in layman's term and to dont show me the corrected code for me to practice fixing the error and I also asked tips on how to debug codes properly.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.